Εθνικό Μετσόβιο Πολυτεχνείο

Σχολή Ηλεκτρολόγων Μηχανικών και Μηχανικών Υπολογιστών

2η Εργαστηριακή Άσκηση

Ασκήσεις στη Βελτιστοποίηση Δυναμικών Δομών Δεδομένων (Dynamic Data Type Refinement – DDTR)

Μάθημα: Σχεδιασμός Ενσωματωμένων Συστημάτων

Εξάμηνο: 9ο

Ονοματεπώνυμα: Μιχαλακόπουλος Δημήτριος (ΑΜ: 03120229), Γκενάκου Ζωή (ΑΜ: 03120015)

Εισαγωγή

Η παρούσα εργαστηριακή άσκηση αποσκοπεί στη διερεύνηση των Δυναμικών Τύπων Δεδομένων (Dynamic Data Types, DDT) και στην αξιολόγηση της βελτιστοποίησής τους μέσω της μεθόδου **Dynamic Data Type Refinement**.

Συγκεκριμένα, θα μελετήσουμε τρία είδη DDT:

- **Απλά Συνδεδεμένη Λίστα** (Single-Linked List - SLL),
- **Διπλά Συνδεδεμένη Λίστα** (Doubly-Linked List - DLL), και
- **Δυναμικό Πίνακα** (Dynamic Array - DYN_ARR).

Η απόδοση των παραπάνω δομών θα αξιολογηθεί με βάση δύο βασικές μετρικές:

1. **Πλήθος Προσβάσεων στη Μνήμη (Memory Accesses)**.
2. **Αποτύπωμα Μνήμης (Memory Footprint)**.

Για τη μελέτη των δομών δεδομένων, θα εφαρμόσουμε τους αλγορίθμους:

- Τον **Deficit Round Robin (DRR)**.
- Τον **αλγόριθμο του Dijkstra**.

Για την εξαγωγή των απαιτούμενων μετρήσεων, χρησιμοποιούμε το εργαλείο **Valgrind**, το οποίο παρέχει πληροφορίες σχετικές με τη χρήση της μνήμης. Συγκεκριμένα:

- Το εργαλείο **Lackey** θα υπολογίσει το συνολικό αριθμό των προσβάσεων στη μνήμη.
- Το εργαλείο **Massif** θα καταγράψει το αποτύπωμα μνήμης.

Κάθε συνδυασμός τύπου δεδομένων μεταγλωττίζεται ξεχωριστά, δημιουργώντας εκτελέσιμα αρχεία ανάλογα με την περίπτωση. Συγκεκριμένα:

- Για τον αλγόριθμο **Deficit Round Robin (DRR)**, παράγονται **9 εκτελέσιμα αρχεία**. Αυτό οφείλεται στο ότι η δομή δεδομένων αφορά τόσο τον πελάτη (node) όσο και το πακέτο (packet).
- Για τον **αλγόριθμο του Dijkstra**, δημιουργούνται **3 εκτελέσιμα αρχεία**, ένα για κάθε τύπο δεδομένων.

Θεωρία

Ακολουθεί μια αναλυτική παρουσίαση των βασικών εννοιών που σχετίζονται με τη μελέτη μας.

Οι **Δυναμικοί Τύποι Δεδομένων (DDT)** είναι θεμελιώδεις για τη διαχείριση δεδομένων σε περιβάλλοντα όπου η δομή της μνήμης δεν είναι στατική και η συμπεριφορά του συστήματος απαιτεί ευέλικτες προσαρμογές. Στο πεδίο των εφαρμογών δικτύου, η ανάγκη για γρήγορη και αποδοτική κατανομή της μνήμης καθιστά τα DDT ιδανική λύση. Τα DDT προσαρμόζονται δυναμικά στις απαιτήσεις των εφαρμογών, καθώς επιτρέπουν τη δυναμική κατανομή και αποδέσμευση μνήμης κατά τη διάρκεια της εκτέλεσης, προσφέροντας έτσι τη δυνατότητα να προσαρμόζονται στις συνεχώς μεταβαλλόμενες ανάγκες των εφαρμογών.

Αντίθετα με τις στατικές δομές δεδομένων, οι οποίες προϋποθέτουν εκ των προτέρων προκαθορισμένο μέγεθος για τη μνήμη, τα DDT επιτρέπουν την αύξηση ή μείωση της καταναλισκόμενης μνήμης, ανάλογα με τις τρέχουσες ανάγκες της εφαρμογής. Σε δίκτυα, όπου οι απαιτήσεις σε δεδομένα και οι λειτουργικές ανάγκες ποικίλλουν, η δυνατότητα προσαρμογής στις απαιτήσεις αυτές είναι ιδιαίτερα σημαντική.

Οι πιο διαδεδομένες δομές DDT που θα εξετάσουμε είναι:

- **Απλά Συνδεδεμένες Λίστες (SLL)**:
  Οι απλές συνδεδεμένες λίστες είναι γραμμικές δομές δεδομένων, όπου κάθε στοιχείο (κόμβος) δείχνει στο επόμενο στοιχείο της ακολουθίας. Αυτή η δομή είναι αποδοτική για λειτουργίες προσθήκης ή διαγραφής στην αρχή της λίστας, αλλά η προσπέλαση των στοιχείων σε πιο προηγμένα σημεία της λίστας είναι πιο αργή, καθώς απαιτεί διαδοχική διέλευση από τον πρώτο προς τον τελευταίο κόμβο.

- **Διπλά Συνδεδεμένες Λίστες (DLL)**:
  Οι διπλά συνδεδεμένες λίστες είναι παρόμοιες με τις απλές, αλλά κάθε κόμβος περιέχει δύο δείκτες: έναν για το επόμενο στοιχείο και έναν για το προηγούμενο. Αυτό επιτρέπει αμφίδρομη κίνηση κατά την περιήγηση στη λίστα, βελτιώνοντας την απόδοση για λειτουργίες που απαιτούν προσπέλαση και από τα δύο άκρα της λίστας.

- **Δυναμικοί Πίνακες (DYN_ARR)**:
  Οι δυναμικοί πίνακες προσφέρουν την ικανότητα αποθήκευσης στοιχείων σε συνεχιζόμενα μπλοκ μνήμης, επιτρέποντας γρήγορη πρόσβαση σε οποιοδήποτε στοιχείο μέσω του δείκτη του. Η κύρια διαφορά τους από τις στατικές δομές δεδομένων είναι η ικανότητα να αυξάνουν ή να μειώνουν το μέγεθός τους, ανάλογα με τις ανάγκες της εφαρμογής. Ωστόσο, η αλλαγή μεγέθους του πίνακα έχει ένα κόστος σε όρους χρόνου και απόδοσης.

Μεθοδολογία Refinement Δυναμικών Δομών Δεδομένων (DDTR) [1]

Η μεθοδολογία **Dynamic Data Type Refinement (DDTR)** αποτελεί μια συστηματική προσέγγιση για τη βελτιστοποίηση των δυναμικών δομών δεδομένων, με στόχο τη βελτίωση της απόδοσης των εφαρμογών δικτύου και την ελαχιστοποίηση της κατανάλωσης πόρων (μνήμη, ενέργεια). Η εφαρμογή αυτής της μεθοδολογίας επιτρέπει στους σχεδιαστές να αναγνωρίσουν και να βελτιώσουν τις παραμέτρους που επηρεάζουν την απόδοση σε πραγματικό χρόνο.

Η μεθοδολογία **DDTR** περιλαμβάνει τρία στάδια:

1. **Application-level DDT Exploration**: Αναγνωρίζονται οι πιο συχνά χρησιμοποιούμενοι τύποι DDT στην εφαρμογή, χρησιμοποιώντας εργαλεία profiling για να αναλυθεί η συμπεριφορά τους σε πραγματικό χρόνο.
2. **Network-level DDT Exploration**: Επεκτείνεται η βελτιστοποίηση σε παραμέτρους δικτύου, εφαρμόζοντας διάφορους τύπους DDT για να βελτιωθεί η απόδοση της εφαρμογής στο δίκτυο.
3. **Pareto-level DDT Exploration**: Παρουσιάζονται οι βέλτιστες λύσεις μέσω καμπυλών Pareto, που απεικονίζουν τα trade-offs μεταξύ μνήμης, χρόνου εκτέλεσης και ενέργειας, διευκολύνοντας τη λήψη τεκμηριωμένων αποφάσεων.

Η μέθοδος αυτή βελτιστοποιεί το σχεδιασμό και μειώνει το χρόνο ανάπτυξης, επιτρέποντας στους μηχανικούς να επιλέξουν την καλύτερη λύση χωρίς πολλές προσομοιώσεις.

Deficit Round Robin

Ο **Deficit Round Robin (DRR)** είναι ένας εξελιγμένος αλγόριθμος δρομολόγησης που εξασφαλίζει τη δίκαιη κατανομή του εύρους ζώνης σε ένα δίκτυο, επιτρέποντας τη σωστή κατανομή της δυναμικής του δικτύου μεταξύ διαφορετικών ροών δεδομένων. Σε αντίθεση με τον παραδοσιακό αλγόριθμο **Round Robin**, ο DRR χρησιμοποιεί έναν μετρητή **deficit** για κάθε ροή, ο οποίος καταγράφει το "χρέος" μιας ροής για τη μη πλήρη χρήση του εύρους ζώνης. Αυτή η τεχνική επιτρέπει τη βέλτιστη εκμετάλλευση των διαθέσιμων πόρων του δικτύου, εξασφαλίζοντας ότι οι μικρότερες ροές δεν παραμελούνται, αλλά έχουν την ευκαιρία να επωφεληθούν από τους αχρησιμοποίητους πόρους όταν είναι διαθέσιμοι.

Dijkstra's Algorithm

Ο **Dijkstra** είναι ένας θεμελιώδης αλγόριθμος εύρεσης της **συντομότερης διαδρομής**, που χρησιμοποιείται ευρέως σε δίκτυα και γραφήματα για να υπολογίσει τη βέλτιστη διαδρομή από έναν αρχικό κόμβο προς όλους τους άλλους κόμβους. Ο αλγόριθμος διατηρεί έναν **πίνακα απόστασης** που καταγράφει το **μικρότερο κόστος** για την προσέγγιση κάθε κόμβου και ανανεώνεται **δυναμικά** καθώς εξετάζονται νέες διαδρομές. Κάθε κόμβος επισκέπτεται μία φορά, και οι γειτονικοί του κόμβοι ενημερώνονται με βάση το σωρευτικό κόστος. Αυτή η διαδικασία διασφαλίζει ότι οι διαδρομές βρίσκονται με αποδοτικό τρόπο, ενώ παράλληλα αποφεύγεται ο υπολογισμός περιττών διαδρομών. Ο αλγόριθμος βρίσκει εφαρμογή σε συστήματα δρομολόγησης και σε εφαρμογές χαρτογράφησης, εξασφαλίζοντας αποτελεσματικότητα και ακρίβεια.

Άσκηση 1: Βελτιστοποίηση δυναμικών δομών δεδομένων του αλγορίθμου DRR

A. Το πρώτο βήμα είναι να εκτελέσουμε όλους τους διαφορετικούς συνδυασμούς υλοποιήσεων δομών δεδομένων για τη λίστα των πακέτων και των κόμβων. Για να το κάνουμε αυτό τροποποιούμε τον κώδικα του αρχείου `drr.h`. Συγκεκριμένα θα τροποποιήσουμε τα πρώτα `#define` και θα διαλέγουμε τον συνδυασμό που θέλουμε να τρέξουμε κάθε φορά.

```
1  #define SLL_CL
2  //#define DLL_CL
3  //#define DYN_ARR_CL
4
5  #define SLL_PK
6  //#define DLL_PK
7  //#define DYN_ARR_PK
```

Για να κάνουμε compile την εφαρμογή εκτελούμε την παρακάτω εντολή:

```
gcc drr.c -o drr -pthread -lcdsl -no-pie -L./../synch_implementations
-I./../synch_implementations
```

Για τα Memory Accesses, δημιουργούμε ένα trace file των προσβάσεων στην μνήμη με χρήση του εργαλείου lackey:

```
valgrind --log-file="mem_accesses_log.txt" --tool=lackey --trace-mem=yes ./drr
```

Καταμετράμε τον αριθμό προσβάσεων στην μνήμη από το trace file:

```
grep -c 'I\| L' mem_accesses_log.txt
```

Για το Memory footprint, δημιουργούμε ένα log αρχείο με χρήση του εργαλείου massif:

```
valgrind --tool=massif ./drr
```

```
ms_print massif.out.XXXXX > mem_footprint_log.txt
```

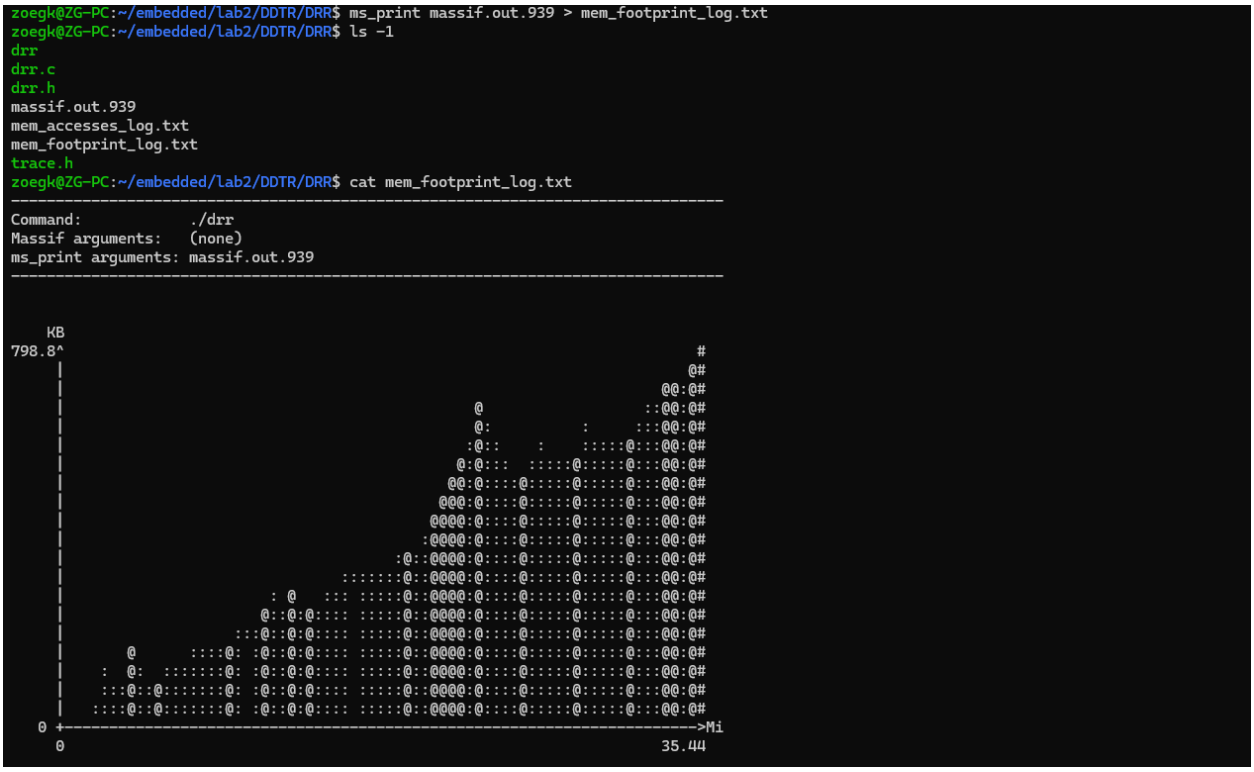
(όπου XXXXX ο αριθμός που αντιστοιχεί στο αρχείο που παράχθηκε).

Στο mem_footprint_log.txt, το memory footprint είναι η μέγιστη τιμή του άξονα y στο διάγραμμα.

Παρακάτω φαίνεται η εκτέλεση των εντολών για την πρώτη περίπτωση:

```
zoegk@ZG-PC:~/embedded/lab2$ cd DDTR/
zoegk@ZG-PC:~/embedded/lab2/DDTR$ ls -1
DRR
dijsktra
synch_implementations
zoegk@ZG-PC:~/embedded/lab2/DDTR$ gcc drr.c -o drr -pthread -lcdsl -no-pie -L./../synch_implementations -I./../synch_implementations
cc1: fatal error: drr.c: No such file or directory
compilation terminated.
zoegk@ZG-PC:~/embedded/lab2/DDTR$ cd DRR
zoegk@ZG-PC:~/embedded/lab2/DDTR/DRR$ gcc drr.c -o drr -pthread -lcdsl -no-pie -L./../synch_implementations -I./../synch_implementations
zoegk@ZG-PC:~/embedded/lab2/DDTR/DRR$ valgrind --log-file="mem_accesses_log.txt" --tool=lackey --trace-mem=yes ./drr
zoegk@ZG-PC:~/embedded/lab2/DDTR/DRR$ grep -c 'I\| L' mem_accesses_log.txt
71866499
zoegk@ZG-PC:~/embedded/lab2/DDTR/DRR$
```

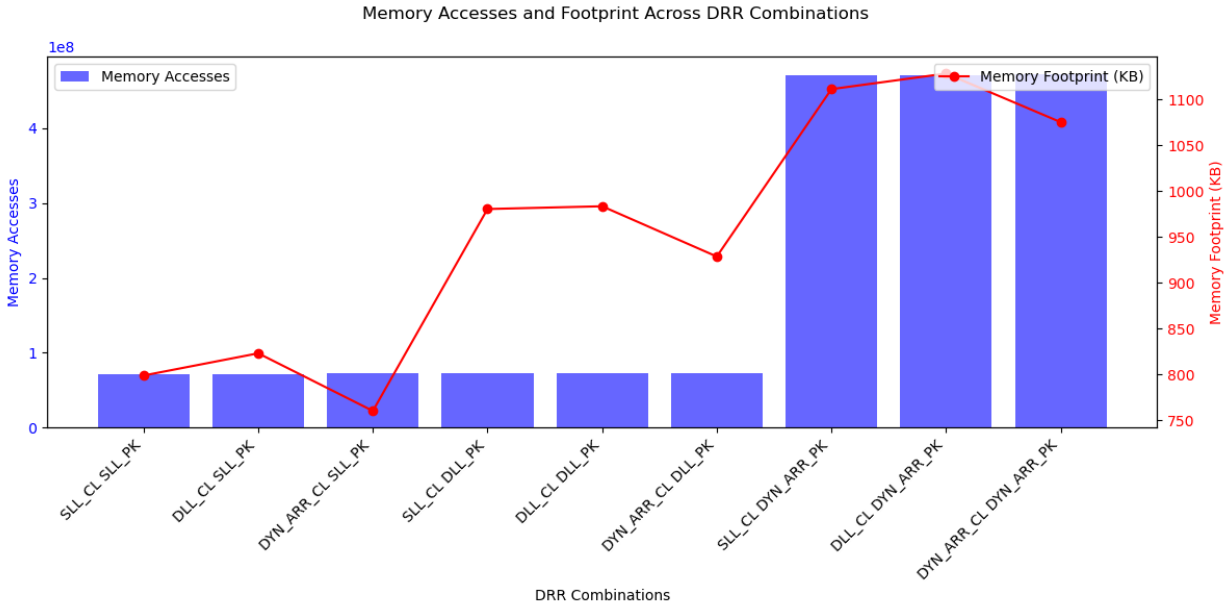
```
zoegk@ZG-PC:~/embedded/lab2/DDTR/DRR$ valgrind --tool=massif ./drr
==939== Massif, a heap profiler
==939== Copyright (C) 2003-2017, and GNU GPL'd, by Nicholas Nethercote
==939== Using Valgrind-3.18.1 and LibVEX; rerun with -h for copyright info
==939== Command: ./drr
==939==
==939==
zoegk@ZG-PC:~/embedded/lab2/DDTR/DRR$
```



Για κάθε συνδυασμό καταγράψαμε τα αποτελέσματά μας σχετικά με τον αριθμό των προσβάσεων στη μνήμη και το μέγεθος της απαιτούμενης μνήμης. Έτσι, προέκυψε ο παρακάτω πίνακας:

Client List	Packet List	Memory Accesses	Memory Footprint
SLL	SLL	71866499	798.8 KB
SLL	DLL	72538713	980.3 KB
SLL	DYN_ARR	470919951	1.111 MB
DLL	SLL	71878478	823.0 KB
DLL	DLL	72550487	983.3 KB
DLL	DYN_ARR	470934985	1.128 MB
DYN_ARR	SLL	72365158	760.2 KB
DYN_ARR	DLL	73050190	928.5 KB
DYN_ARR	DYN_ARR	471667914	1.075 MB

Παρακάτω φαίνεται μια εικονική αναπαράσταση των αποτελεσμάτων ως bar plot και line plot.



Η δομή δεδομένων του **packet list** επηρεάζει σημαντικά τόσο τις προσπελάσεις στη μνήμη όσο και το αποτύπωμα μνήμης. Η δομή του **client list**, από την άλλη, έχει μικρότερη επίδραση, όπως φαίνεται από την ομοιότητα στις τιμές όταν η packet list μένει σταθερή. Η χρήση δυναμικού πίνακα στο packet list αυξάνει δραστικά τις προσπελάσεις μνήμης (σχεδόν κατά μια τάξη μεγέθους) και το αποτύπωμα μνήμης, επιβεβαιώνοντας ότι δεν είναι κατάλληλη επιλογή για αποδοτικότητα.

B. Η ανάλυση των προσβάσεων μνήμης αποκαλύπτει τη σημασία των επιλογών δυναμικών δομών δεδομένων (Dynamic Data Types - DDT) τόσο για το **client list** όσο και για το **packet list** στον αλγόριθμο **Deficit Round Robin (DRR)**.

- Οι επιλογές για το client list, όπως αναφέραμε, δεν επηρεάζουν δραστικά τις προσπελάσεις μνήμης. Είτε χρησιμοποιηθεί **Single Linked List (SLL)**, **Double Linked List (DLL)**, είτε **Dynamic Array (DYN_ARR)**, η διαφορά στις προσβάσεις μνήμης είναι μικρή. Ωστόσο, το DYN_ARR παρουσιάζει ελαφρώς αυξημένες προσβάσεις λόγω της στατικής φύσης των πινάκων και του κόστους επανακατανομής τους.

- Η δομή του packet list έχει καθοριστικό ρόλο στις προσβάσεις μνήμης.
  - **SLL:** Προσφέρει τις χαμηλότερες τιμές, με τις προσβάσεις να φτάνουν περίπου τις **71866499**. Η αποδοτικότητα αυτή οφείλεται στη γραμμική φύση της λίστας και την απλότητα διαχείρισης της μνήμης.
  - **DLL:** Αν και παρέχει διπλή συνδεσιμότητα, οι προσβάσεις μνήμης είναι ελαφρώς αυξημένες σε σχέση με την SLL, χωρίς ουσιαστικά οφέλη σε αυτό το πλαίσιο.

Αυτό οφείλεται στη χρήση επιπλέον δείκτη (pointer), ο οποίος, αν και ενισχύει τη λειτουργικότητα, δεν μειώνει το πλήθος των προσβάσεων.
    - **DYN_ARR:** Η χρήση του οδηγεί σε υπερβολικά αυξημένες προσβάσεις μνήμης, φτάνοντας έως και **471 εκατομμύρια**. Αυτό το κόστος οφείλεται στη στατική δέσμευση και την επανακατανομή μνήμης όταν ο πίνακας χρειάζεται επέκταση.

Η βέλτιστη επιλογή για τη μείωση των προσπελάσεων μνήμης είναι:

- **Client List:** Single Linked List (SLL)
- **Packet List:** Single Linked List (SLL)
- **Memory Accesses:** 71866499

C. Η κατανάλωση μνήμης (Memory Footprint) επηρεάζεται και από τις δύο δομές, με τη σημαντικότερη επιρροή να προέρχεται από το **packet list**.

- **Client List:**
    - Η **Dynamic Array (DYN_ARR)** αποδεικνύεται η πιο αποδοτική επιλογή, με το αποτύπωμα μνήμης να μειώνεται στις **760.2 KB** όταν συνδυάζεται με SLL στο packet list. Αυτό οφείλεται στη συνεχή κατανομή μνήμης και την απουσία επιπλέον συνδέσμων.
    - Αντίθετα, οι SLL και DLL στο client list αυξάνουν το αποτύπωμα μνήμης λόγω της προσθήκης δεικτών.

- **Packet List:**
    - **SLL:** Παρουσιάζει το χαμηλότερο αποτύπωμα μνήμης, με τιμές που κυμαίνονται κοντά στις **760-798 KB** ανάλογα με τη δομή του client list.
    - **DLL:** Το αποτύπωμα αυξάνεται, φτάνοντας τα **980 KB**, λόγω του διπλού δείκτη που απαιτείται για την υποστήριξη αμφίδρομης συνδεσιμότητας.
    - **DYN_ARR:** Παράγει το υψηλότερο αποτύπωμα μνήμης, ξεπερνώντας το **1 MB**, κυρίως λόγω της ανάγκης επανακατανομής και της συνεχούς κατανομής μνήμης.

Η τάση είναι γραμμική: καθώς πηγαίνουμε από **SLL → DLL → DYN_ARR** στο packet list, το αποτύπωμα μνήμης αυξάνεται σταθερά.

Η βέλτιστη επιλογή για τη μείωση του αποτυπώματος μνήμης είναι:

- **Client List:** Dynamic Array (DYN_ARR)
- **Packet List:** Single Linked List (SLL)
- **Memory Footprint:** 760.2 KB

Άσκηση 2: Βελτιστοποίηση δυναμικών δομών δεδομένων του αλγορίθμου Dijkstra

A. Το πρώτο βήμα είναι να εκτελέσουμε τον αλγόριθμο του Dijkstra, όπως μας δίνεται έτοιμος, ο οποίος βρίσκεται στο αρχείο `dijkstra.c` με τα δεδομένα του αρχείου `input.dat`, χωρίς να έχουμε κάνει εισαγωγή της βιβλιοθήκης **DDTR**.

Για να κάνουμε compile και να τρέξουμε την εφαρμογή εκτελούμε τις παρακάτω εντολές:

```
gcc dijkstra.c -o dijkstra -pthread
./dijkstra input.dat
```

Το αποτέλεσμα εκτέλεσης της εφαρμογής φαίνεται στην παρακάτω εικόνα:

```
zoegk@ZG-PC:~/embedded/lab2/DDTR/dijsktra$ gcc dijkstra.c -o dijkstra -pthread
zoegk@ZG-PC:~/embedded/lab2/DDTR/dijsktra$ ./dijkstra input.dat
Shortest path is 1 in cost. Path is:  0 41 45 51 50
Shortest path is 0 in cost. Path is:  1 58 57 20 40 17 65 73 36 46 10 38 41 45 51
Shortest path is 1 in cost. Path is:  2 71 47 79 23 77 1 58 57 20 40 17 52
Shortest path is 2 in cost. Path is:  3 53
Shortest path is 1 in cost. Path is:  4 85 83 58 33 13 19 79 23 77 1 54
Shortest path is 3 in cost. Path is:  5 26 23 77 1 58 99 3 21 70 55
Shortest path is 3 in cost. Path is:  6 42 80 77 1 58 99 3 21 70 55 56
Shortest path is 0 in cost. Path is:  7 17 65 73 36 46 10 58 57
Shortest path is 0 in cost. Path is:  8 37 63 72 46 10 58
Shortest path is 1 in cost. Path is:  9 33 13 19 79 23 77 1 59
Shortest path is 0 in cost. Path is:  10 60
Shortest path is 5 in cost. Path is:  11 22 20 40 17 65 73 36 46 10 29 61
Shortest path is 0 in cost. Path is:  12 37 63 72 46 10 58 99 3 21 70 62
Shortest path is 0 in cost. Path is:  13 19 79 23 77 1 58 99 3 21 70 55 12 37 63
Shortest path is 1 in cost. Path is:  14 38 41 45 51 68 2 71 47 79 23 77 1 58 33 13 92 64
Shortest path is 1 in cost. Path is:  15 13 92 94 11 22 20 40 17 65
Shortest path is 3 in cost. Path is:  16 41 45 51 68 2 71 47 79 23 77 1 58 33 32 66
Shortest path is 0 in cost. Path is:  17 65 73 36 46 10 58 33 13 19 79 23 91 67
Shortest path is 1 in cost. Path is:  18 15 41 45 51 68
Shortest path is 2 in cost. Path is:  19 69
```

B. Το επόμενο βήμα είναι να κάνουμε εισαγωγή της βιβλιοθήκης **DDTR** στην υλοποίηση του αλγορίθμου, επιλέγοντας ως δομές δεδομένων την SLL, την DLL και την DYN_ARR, όπως κάναμε και για τον αλγόριθμο DDR. Προφανώς δεν αρκεί να κάνουμε define τα απαραίτητα header files, αλλά πρέπει να πραγματοποιήσουμε και αλλαγές στον κώδικα. Η διαδικασία που ακολουθήσαμε, καθώς και οι αλλαγές που υπέστη ο κώδικας είναι οι παρακάτω:

- Αρχικά, κάνουμε **define και include τα header files** `cdsl_queue.h, cdsl_deque.h` και `cdsl_dyn_array.h`, που περιέχουν τις δηλώσεις των συναρτήσεων (function declarations) και τις δομές (structs) που υλοποιούν τις ζητούμενες δυναμικές δομές δεδομένων της DDTR βιβλιοθήκης που θα χρησιμοποιήσουμε.

```c
#include <stdio.h>

#define SLL
//#define DLL
//#define DYN_ARR

#if defined(SLL)
#include "../synch_implementations/cdsl_queue.h"
#endif
#if defined(DLL)
#include "../synch_implementations/cdsl_deque.h"
#endif
#if defined(DYN_ARR)
#include "../synch_implementations/cdsl_dyn_array.h"
#endif

#define NUM_NODES 100
#define NONE 9999
```

- Έπειτα, ανάλογα με ποια δομή δεδομένων είναι ορισμένη, **δηλώνουμε τις νέες μεταβλητές** της βιβλιοθήκης DDTR, αντικαθιστώντας έτσι τις δηλώσεις των αρχικών δομών δεδομένων. Σημαντικό είναι να αναφερθεί ότι δηλώνουμε και έναν **iterator** σε κάθε περίπτωση, ο οποίος χρησιμοποιείται αργότερα στην τροποποίηση της συνάρτησης dequeue.

```c
struct _QITEM
{
    int iNode;
    int iDist;
    int iPrev;
};
typedef struct _QITEM QITEM;

#if defined(SLL)
cdsl_sll *qHead;
iterator_cdsl_sll it;
#endif
#if defined(DLL)
cdsl_dll *qHead;
iterator_cdsl_dll it;
#endif
#if defined(DYN_ARR)
cdsl_dyn_array *qHead;
iterator_cdsl_dyn_array it;
#endif

int AdjMatrix[NUM_NODES][NUM_NODES];
```

- Στην συνέχεια, πηγαίνουμε στην main συνάρτηση και **αρχικοποιούμε** κάθε δομή δεδομένων καλώντας αντίστοιχα τις συναρτήσεις `cdsl_sll_init()`, `cdsl_dll_init()` και `cdsl_dyn_array_init()`.

```
169     #if defined(SLL)
170     qHead = cdsl_sll_init();
171     #endif
172     #if defined(DLL)
173     qHead = cdsl_dll_init();
174     #endif
175     #if defined(DYN_ARR)
176     qHead = cdsl_dyn_array_init();
177     #endif
178
```

- Τέλος, **τροποποιούμε** τις συναρτήσεις **enqueue** και **dequeue** κατάλληλα, ώστε η προσθήκη και η αφαίρεση στοιχείων στην ουρά δεδομένων (queue) να λειτουργούν σωστά σύμφωνα με τις νέες δομές δεδομένων που εισήγαμε μέσω της βιβλιοθήκης DDTR.

```
66  void enqueue(int iNode, int iDist, int iPrev)
67  {
68    QITEM *qNew = (QITEM *)malloc(sizeof(QITEM));
69
70    if (!qNew)
71    {
72      fprintf(stderr, "Out of memory.\n");
73      exit(1);
74    }
75    qNew->iNode = iNode;
76    qNew->iDist = iDist;
77    qNew->iPrev = iPrev;
78
79    qHead->enqueue(0, qHead, (void *)qNew);
80    g_qCount++;
81  }
82
83  void dequeue(int *piNode, int *piDist, int *piPrev)
84  {
85    it = qHead->iter_begin(qHead);
86    QITEM *item = (QITEM*)(qHead->iter_deref(qHead, it));
87
88    *piNode = item->iNode;
89    *piDist = item->iDist;
90    *piPrev = item->iPrev;
91
92    qHead->dequeue(0, (qHead));
93    g_qCount--;
94  }
```

Εκτελώντας **ξανά** την εφαρμογή παίρνουμε τα ίδια αποτελέσματα συγκριτικά με το ερώτημα (a), επομένως έχουμε **εισάγει επιτυχώς** την βιβλιοθήκη DDTR στον κώδικά μας.

```
zoegk@ZG-PC:~/embedded/lab2/DDTR/dijsktra$ gcc dijkstra.c -o dijkstra -pthread -lcdsl -no-pie -L./../synch_implementations -I./../synch_implementations
zoegk@ZG-PC:~/embedded/lab2/DDTR/dijsktra$ ./dijkstra input.dat
Shortest path is 1 in cost. Path is:  0 41 45 51 50
Shortest path is 0 in cost. Path is:  1 58 57 20 40 17 65 73 36 46 10 38 41 45 51
Shortest path is 1 in cost. Path is:  2 71 47 79 23 77 1 58 57 20 40 17 52
Shortest path is 2 in cost. Path is:  3 53
Shortest path is 1 in cost. Path is:  4 85 83 58 33 13 19 79 23 77 1 54
Shortest path is 3 in cost. Path is:  5 26 23 77 1 58 99 3 21 70 55
Shortest path is 3 in cost. Path is:  6 42 80 77 1 58 99 3 21 70 55 56
Shortest path is 0 in cost. Path is:  7 17 65 73 36 46 10 58 57
Shortest path is 0 in cost. Path is:  8 37 63 72 46 10 58
Shortest path is 1 in cost. Path is:  9 33 13 19 79 23 77 1 59
Shortest path is 0 in cost. Path is:  10 60
Shortest path is 5 in cost. Path is:  11 22 20 40 17 65 73 36 46 10 29 61
Shortest path is 0 in cost. Path is:  12 37 63 72 46 10 58 99 3 21 70 62
Shortest path is 0 in cost. Path is:  13 19 79 23 77 1 58 99 3 21 70 55 12 37 63
Shortest path is 1 in cost. Path is:  14 38 41 45 51 68 2 71 47 79 23 77 1 58 33 13 92 64
Shortest path is 1 in cost. Path is:  15 13 92 94 11 22 20 40 17 65
Shortest path is 3 in cost. Path is:  16 41 45 51 68 2 71 47 79 23 77 1 58 33 32 66
Shortest path is 0 in cost. Path is:  17 65 73 36 46 10 58 33 13 19 79 23 91 67
Shortest path is 1 in cost. Path is:  18 15 41 45 51 68
Shortest path is 2 in cost. Path is:  19 69
zoegk@ZG-PC:~/embedded/lab2/DDTR/dijsktra$
```

C. Σε αυτό το ερώτημα θέλουμε να εκτελέσουμε την εφαρμογή μας για τις τρεις διαφορετικές δυναμικές δομές δεδομένων που αναφερθήκαμε και εισήγαμε προηγουμένως. Για κάθε μία υλοποίηση καταγράφουμε τον **αριθμό προσβάσεων στην μνήμη** (memory accesses) και το **μέγιστο μέγεθος μνήμης** (memory footprint).

Για να κάνουμε compile την εφαρμογή εκτελούμε την παρακάτω εντολή:

```
gcc dijkstra.c -o dijkstra -pthread -lcdsl -no-pie -
L./../synch_implementations -I./../synch_implementations
```

Για τα Memory Accesses, δημιουργούμε ένα trace file των προσβάσεων στην μνήμη με χρήση του εργαλείου lackey:

```
valgrind --log-file="mem_accesses_log.txt" --tool=lackey --trace-
mem=yes ./dijkstra input.dat
```

Καταμετράμε τον αριθμό προσβάσεων στην μνήμη από το trace file:

```
grep -c 'I\| L' mem_accesses_log.txt
```

Για το Memory footprint, δημιουργούμε ένα log αρχείο με χρήση του εργαλείου massif:

```
valgrind --tool=massif ./dijkstra input.dat
```

```
ms_print massif.out.XXXXX > mem_footprint_log.txt
```

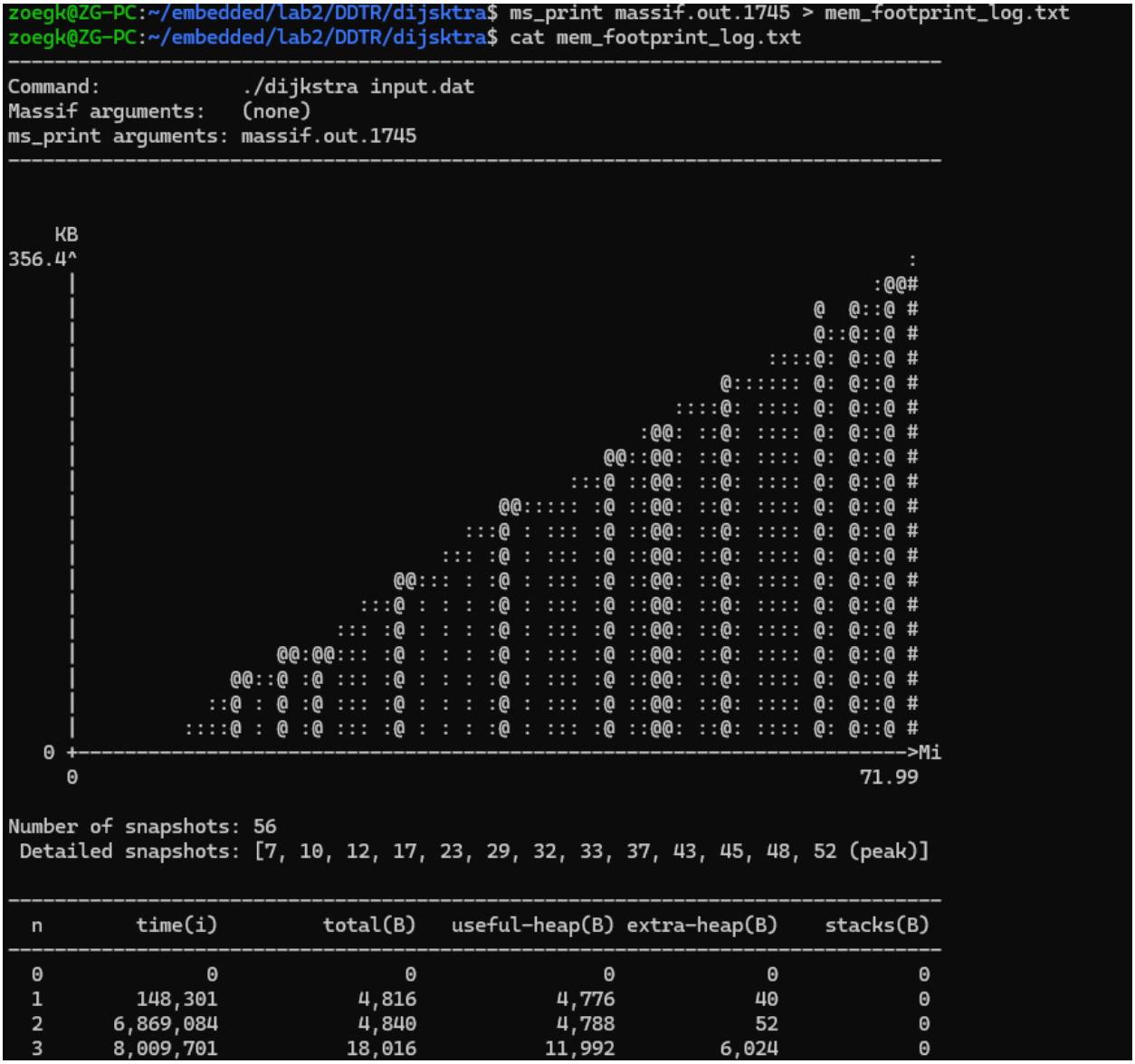
(όπου XXXXX ο αριθμός που αντιστοιχεί στο αρχείο που παράχθηκε).

Στο mem_footprint_log.txt, το memory footprint είναι η μέγιστη τιμή του άξονα y στο διάγραμμα.

Παρακάτω φαίνεται η εκτέλεση των εντολών για την πρώτη περίπτωση:

```
zoegk@ZG-PC:~/embedded/lab2/DDTR/dijsktra$ gcc dijkstra.c -o dijkstra -pthread -lcdsl -no-pie -L./../synch_implementations -I./../synch_implementations
zoegk@ZG-PC:~/embedded/lab2/DDTR/dijsktra$ ./dijkstra input.dat
Shortest path is 1 in cost. Path is:  0 41 45 51 50
Shortest path is 0 in cost. Path is:  1 58 57 20 40 17 65 73 36 46 10 38 41 45 51
Shortest path is 1 in cost. Path is:  2 71 47 79 23 77 1 58 57 20 40 17 52
Shortest path is 2 in cost. Path is:  3 53
Shortest path is 1 in cost. Path is:  4 85 83 58 33 13 19 79 23 77 1 54
Shortest path is 3 in cost. Path is:  5 26 23 77 1 58 99 3 21 70 55
Shortest path is 3 in cost. Path is:  6 42 80 77 1 58 99 3 21 70 55 56
Shortest path is 0 in cost. Path is:  7 17 65 73 36 46 10 58 57
Shortest path is 0 in cost. Path is:  8 37 63 72 46 10 58
Shortest path is 1 in cost. Path is:  9 33 13 19 79 23 77 1 59
Shortest path is 0 in cost. Path is:  10 60
Shortest path is 5 in cost. Path is:  11 22 20 40 17 65 73 36 46 10 29 61
Shortest path is 0 in cost. Path is:  12 37 63 72 46 10 58 99 3 21 70 62
Shortest path is 0 in cost. Path is:  13 19 79 23 77 1 58 99 3 21 70 55 12 37 63
Shortest path is 1 in cost. Path is:  14 38 41 45 51 68 2 71 47 79 23 77 1 58 33 13 92 64
Shortest path is 1 in cost. Path is:  15 13 92 94 11 22 20 40 17 65
Shortest path is 3 in cost. Path is:  16 41 45 51 68 2 71 47 79 23 77 1 58 33 32 66
Shortest path is 0 in cost. Path is:  17 65 73 36 46 10 58 33 13 19 79 23 91 67
Shortest path is 1 in cost. Path is:  18 15 41 45 51 68
Shortest path is 2 in cost. Path is:  19 69
zoegk@ZG-PC:~/embedded/lab2/DDTR/dijsktra$
```

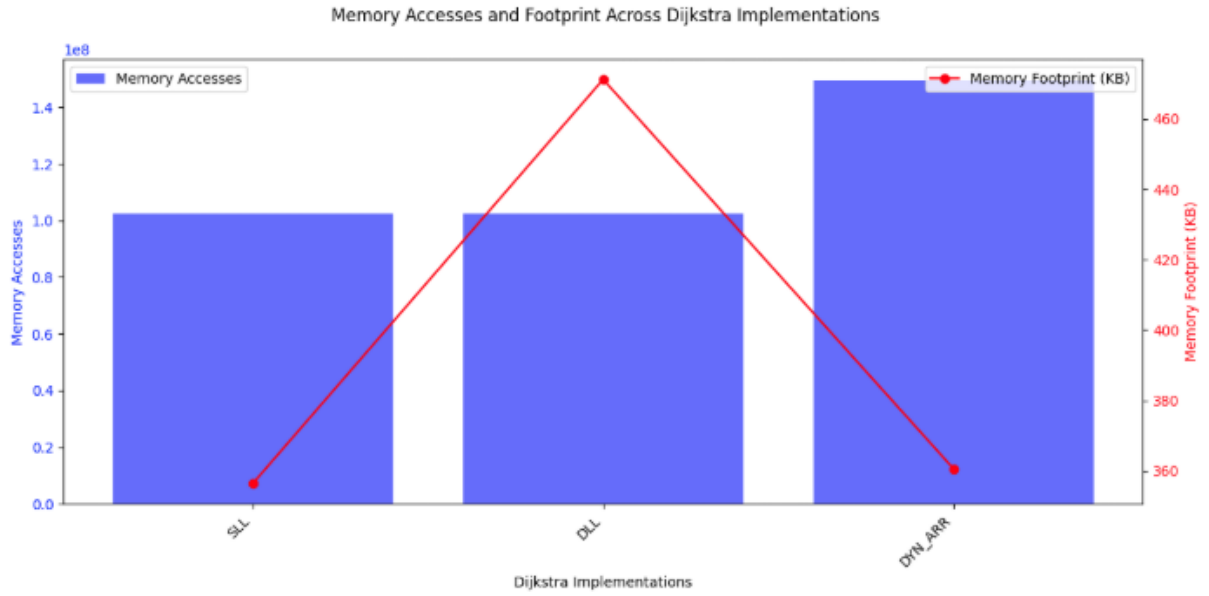
```
zoegk@ZG-PC:~/embedded/lab2/DDTR/dijsktra$ valgrind --log-file="mem_accesses_log.txt" --tool=lackey --trace-mem=yes ./dijkstra input.dat
Shortest path is 1 in cost. Path is:  0 41 45 51 50
Shortest path is 0 in cost. Path is:  1 58 57 20 40 17 65 73 36 46 10 38 41 45 51
Shortest path is 1 in cost. Path is:  2 71 47 79 23 77 1 58 57 20 40 17 52
Shortest path is 2 in cost. Path is:  3 53
Shortest path is 1 in cost. Path is:  4 85 83 58 33 13 19 79 23 77 1 54
Shortest path is 3 in cost. Path is:  5 26 23 77 1 58 99 3 21 70 55
Shortest path is 3 in cost. Path is:  6 42 80 77 1 58 99 3 21 70 55 56
Shortest path is 0 in cost. Path is:  7 17 65 73 36 46 10 58 57
Shortest path is 0 in cost. Path is:  8 37 63 72 46 10 58
Shortest path is 1 in cost. Path is:  9 33 13 19 79 23 77 1 59
Shortest path is 0 in cost. Path is:  10 60
Shortest path is 5 in cost. Path is:  11 22 20 40 17 65 73 36 46 10 29 61
Shortest path is 0 in cost. Path is:  12 37 63 72 46 10 58 99 3 21 70 62
Shortest path is 0 in cost. Path is:  13 19 79 23 77 1 58 99 3 21 70 55 12 37 63
Shortest path is 1 in cost. Path is:  14 38 41 45 51 68 2 71 47 79 23 77 1 58 33 13 92 64
Shortest path is 1 in cost. Path is:  15 13 92 94 11 22 20 40 17 65
Shortest path is 3 in cost. Path is:  16 41 45 51 68 2 71 47 79 23 77 1 58 33 32 66
Shortest path is 0 in cost. Path is:  17 65 73 36 46 10 58 33 13 19 79 23 91 67
Shortest path is 1 in cost. Path is:  18 15 41 45 51 68
Shortest path is 2 in cost. Path is:  19 69
zoegk@ZG-PC:~/embedded/lab2/DDTR/dijsktra$ grep -c 'I\| L' mem_accesses_log.txt
102254663
zoegk@ZG-PC:~/embedded/lab2/DDTR/dijsktra$
```



Για κάθε υλοποίηση καταγράψαμε τα αποτελέσματά μας σχετικά με τον αριθμό των προσβάσεων στη μνήμη και το μέγεθος της απαιτούμενης μνήμης. Έτσι, προέκυψε ο παρακάτω πίνακας:

Data Structure	Memory Accesses	Memory Footprint
SLL	102254663	356.4 KB
DLL	102433847	471.2 KB
DYN_ARR	149469215	360.6 KB

Παρακάτω φαίνεται μια εικονική αναπαράσταση των αποτελεσμάτων ως bar plot και line plot.



D. Η υλοποίηση δομής δεδομένων, η οποία πετυχαίνει τον **μικρότερο αριθμό προσβάσεων στην μνήμη** είναι η **SLL** (Single Linked List) με 102254663 memory accesses. Συγκριτικά, η υλοποίηση με DLL πραγματοποιεί 102433847 memory accesses, οι οποίες είναι ελάχιστα περισσότερες σε σχέση με την SLL, ενώ η υλοποίηση με DYN_ARR εκτελεί 149469215 memory accesses, οι οποίες είναι υπερβολικά αυξημένες. Η SLL, λοιπόν, είναι ταχύτερη λόγω της συνεχούς μνήμης, μεγιστοποιώντας την αποδοτικότητα της κρυφής μνήμης και εξαλείφοντας την υπερφόρτωση δεικτών. Το DLL υποφέρει από κατακερματισμένη μνήμη και συχνές αποτυχίες κρυφής μνήμης (cache misses), ενώ το DYN_ARR είναι πιο αργό λόγω της αλλαγής μεγέθους (resizing) και της ενδεχόμενης κατακερματισμένης μνήμης.

E. Η υλοποίηση δομής δεδομένων, η οποία έχει **μικρότερες απαιτήσεις σε μνήμη** είναι η **SLL** (Single Linked List) με memory footprint 356.4 KB. Συγκριτικά, η υλοποίηση με DLL έχει memory footprint 471.2 KB, το οποίο είναι σημαντικά μεγαλύτερο από αυτό του SLL, ενώ η υλοποίηση με DYN_ARR έχει memory footprint 360.6 KB, που είναι ελάχιστα μεγαλύτερο από του SLL. Η SLL, λοιπόν, έχει μικρότερο αποτύπωμα μνήμης από τη DYN_ARR, καθώς αποθηκεύει μόνο τα δεδομένα και έναν μόνο δείκτη ανά κόμβο, ενώ η DYN_ARR δεσμεύει επιπλέον χώρο για αναδιάρθρωση, προκαλώντας πιθανή σπατάλη μνήμης. Η SLL ξεπερνά επίσης τη DLL, που απαιτεί δύο δείκτες ανά κόμβο, πολλαπλασιάζοντας τη χρήση μνήμης.

Το script που υλοποιεί το γράφημα της Άσκησης 1 βρίσκεται στο αρχείο `plot1.py`.
Το script που υλοποιεί το γράφημα της Άσκησης 2 βρίσκεται στο αρχείο `plot2.py`.
Ο τροποποιημένος κώδικας του αλγόριθμου Dijkstra βρίσκεται στο αρχείο `dijkstra.c`.
Τα αποτελέσματα των εκτελέσεων βρίσκονται στο αρχείο `values.txt`.